Mise en place d' une infrastructure PKI

Le projet consiste à construire une infrastructure de confiance numérique appelée **PKI**.
PKI est l'abréviation de **Public Key Infrastructure** (**Infrastructure à clés publiques**)

C'est un ensemble de systèmes, de règles et de mécanismes qui permettent de créer, distribuer, vérifier et révoquer des certificats numériques.

Par exemple, lorsq'on visites un site comme **https://www.google.com**
Le navigateur vérifie que le site possède un certificat numérique valide.
Ce certificat permet notamment de vérifier :

- Qu'on communiques avec le bon serveur.
- Que le serveur possède bien l'identité annoncée dans son certificat.
- Que les communications entre lr navigateur et le serveur sont chiffrées.
- Que le certificat a été délivré par une autorité de certification reconnue.

Certificat numérique - Permet de vérifier l'identité d'un serveur, d'un utilisateur ou d'un autre équipement informatique. Et est utilisé par les ordinateurs pour établir une relation de confiance.

Mais qui fabrique ces certificats ? Qui garantit qu'ils sont fiables ? Comment peut-on les annuler lorsqu'ils sont compromis ?

Pourquoi une entreprise a-t-elle besoin d'une PKI ?
Imaginons une entreprise qui possède :
Un serveur web.
Un serveur de messagerie.
Des ordinateurs pour ses employés.
Des applications internes.
Des serveurs de bases de données.

Ces équipements communiquent régulièrement entre eux. Mais comment être certain que :
Le serveur auquel on se connecte est bien le bon ?
Personne ne se fait passer pour un serveur légitime ?
Les communications ne peuvent pas être facilement interceptées et lues ?
Seuls les utilisateurs autorisés peuvent accéder à certaines applications ?

Une **PKI** permet de répondre à une grande partie de ces problèmes.
Elle permet de mettre en place :

HTTPS ->	Sécuriser un site web
Authentification des serveurs ->	Vérifier l'identité d'un serveur
Authentification des utilisateurs ->	Vérifier l'identité d'un utilisateur
Chiffrement ->	Protéger les communications
Signature numérique ->	Vérifier l'authenticité et l'intégrité d'un document
Révocation ->	Invalider un certificat compromis

Comprendre les termes importants:

Le chiffrement consiste à transformer un message lisible en un message illisible pour les personnes qui ne possèdent pas la bonne clé. Le chiffrement sert principalement à protéger la confidentialité des informations.

Clés cryptographiques Une clé cryptographique est une information utilisée pour effectuer des opérations de chiffrement, de déchiffrement ou de signature numérique.
La clé privée
Elle doit rester secrète.
Elle appartient à une personne, un serveur ou une autorité de certification.
Elle ne doit jamais être communiquée à n'importe qui.
La clé publique
Elle peut être communiquée librement.
Elle est mathématiquement liée à la clé privée.
Elle permet notamment de vérifier les signatures numériques.

On appelle cela la **cryptographie asymétrique**, parce qu'elle utilise deux clés différentes qui fonctionnent ensemble.

Le certificat numérique est un document électronique qui associe une identité à une clé publique.
Il contient:
Le nom ou l'identité du propriétaire.
Sa clé publique.
La période de validité.
Le nom de l'autorité qui l'a délivré.
Son numéro de série.

Sa signature numérique.

Le format appelé **X.509** c'est le standard utilisé pour les certificats TLS/SSL.

L'autorité de certification (CA - Certificate Authority) C'est une organisation ou un système informatique qui délivre et signe les certificats numériques.

La CA racine (Root Certificate Authority) est l'autorité de certification située tout en haut de notre infrastructure.
C'est la principale autorité de confiance.
Elle possède :

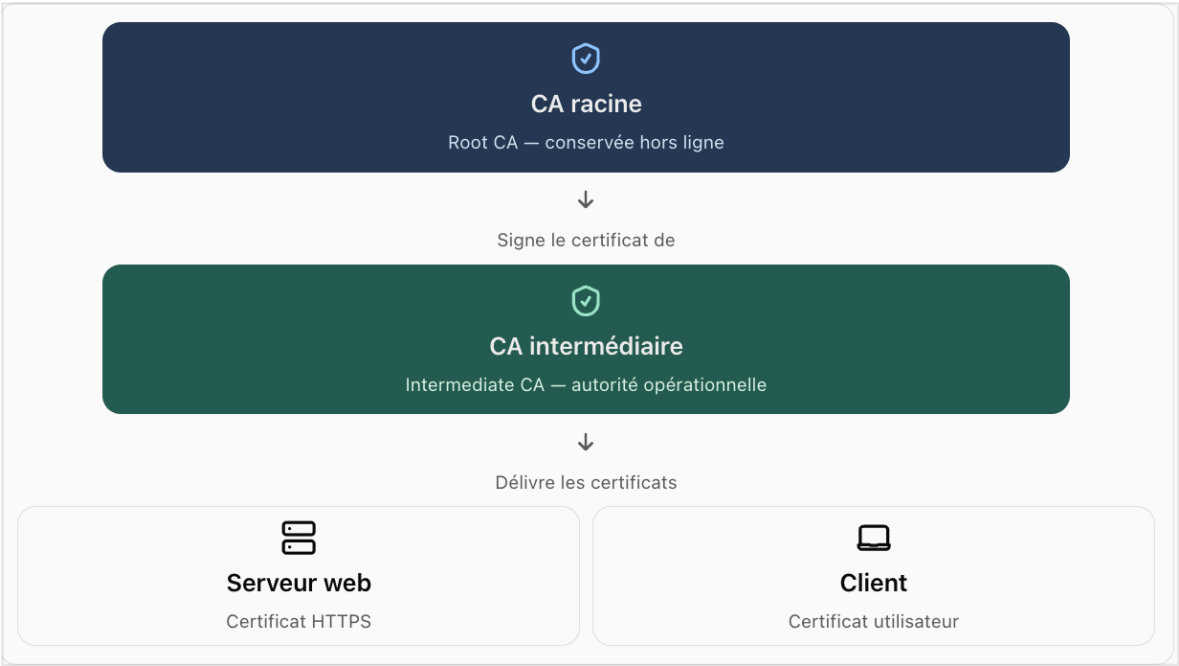
- Une clé privée.
- Un certificat racine.

Elle sert à signer le certificat de l'autorité intermédiaire.

La clé privée de la **CA racine** doit être particulièrement protégée.

La CA intermédiaire (Intermediate Certificate Authority) est une deuxième autorité de certification. Elle est signée par la **CA racine**. Son rôle est de délivrer les certificats aux serveurs et aux utilisateurs.

Pourquoi ne pas utiliser directement la **CA racine** ? Pour des raisons de sécurité.
Si notre **CA intermédiaire** est compromise, nous pouvons la révoquer et en créer une nouvelle sans devoir systématiquement remplacer la **CA racine**.



Cette organisation s'appelle une **chaîne de confiance**.

La signature numérique Lorsque notre **CA intermédiaire** délivre un certificat, elle le signe avec sa clé privée. Cette signature permet de vérifier que le certificat a bien été délivré par cette autorité et qu'il n'a pas été modifié. La signature sert à garantir son authenticité et son intégrité.

SSL est un ancien protocole de sécurisation des communications. (Secure Sockets Layer (Couche de sockets sécurisée)).
TLS est son successeur moderne. (Transport Layer Security (Sécurité de la couche transport)).
HTTPS est HTTP utilisé avec TLS. (HyperText Transfer Protocol Secure (Protocole de transfert hypertexte sécurisé)).

Aujourd'hui, on utilise TLS et non les anciennes versions de SSL. SSL est aujourd'hui obsolète et n'est plus considéré comme sécurisé.
Lorsqu'on visites un site en HTTPS, TLS permet notamment de protéger les échanges entre le navigateur et le serveur.

Dans notre projet, nous allons utiliser TLS pour sécuriser notre propre site web.

OpenSSL est un logiciel et une bibliothèque utilisés pour les opérations cryptographiques.
Il permet de :

- Générer des clés privées et publiques.
- Créer des certificats numériques.
- Créer une autorité de certification (CA).
- Signer des certificats.

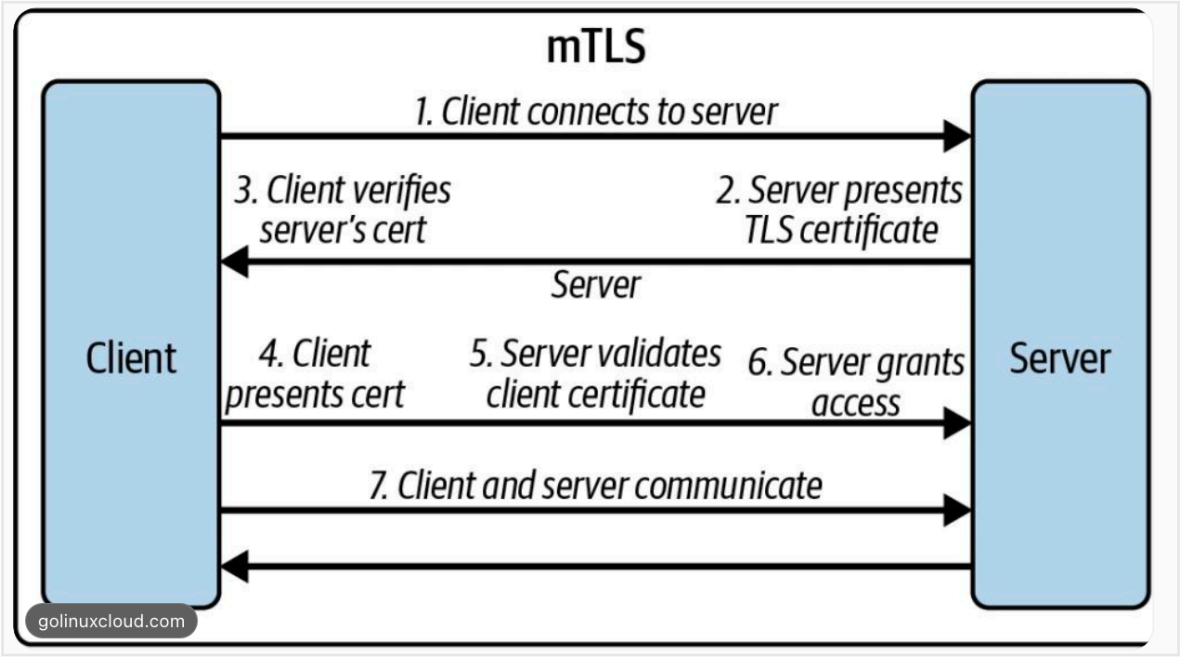
- Vérifier des certificats.
- Chiffrer et déchiffrer des données.
- Gérer des opérations cryptographiques liées à TLS.
- Générer des listes CRL.

Nginx Un serveur web qui permet d'héberger des sites et de gérer les connexions HTTPS.
Nous allons configurer **Nginx** pour :

- Utiliser le certificat serveur délivré par notre CA (Certificate Authority)
- Activer HTTPS.
- Demander un certificat au client.
- Vérifier que ce certificat est valide.
- Refuser les certificats révoqués.

L'authentification mutuelle (mTLS) **mTLS** signifie **Mutual TLS** authentification TLS mutuelle.
TLS classique :
Le serveur présente son certificat au client **->** *Le client vérifie l'identité du serveur.*

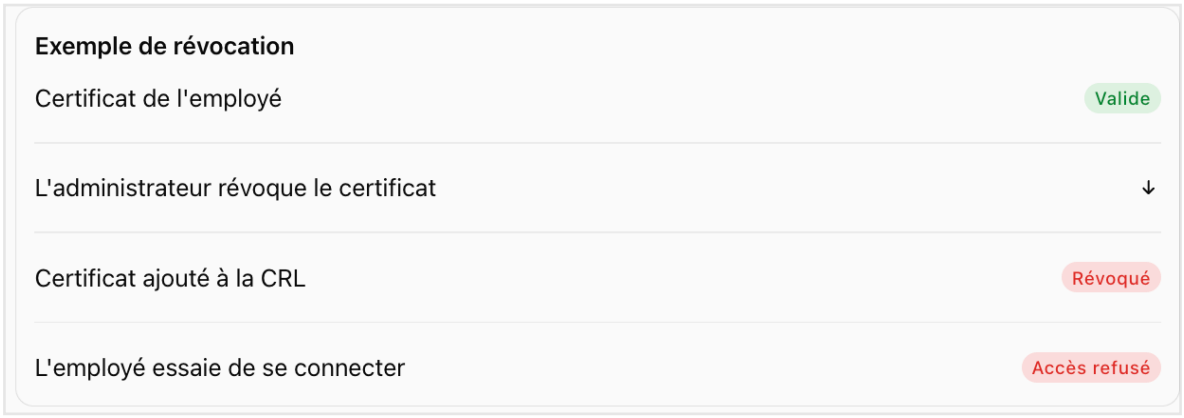
Cette fois, les deux parties présentent un certificat :
Le serveur présente son certificat au client **->** *Le client présente son certificat au serveur.*
Le serveur vérifie que le certificat du client a été délivré par notre **CA**.



La révocation d'un certificat l'action d'invalider un certificat avant même sa date d'expiration et le certificat ne doit plus permettre d'accéder au site web.

Il existe deux mécanismes importants:
CRL Certificate Revocation List c'est une liste des certificats qui ont été révoqués. Le serveur peut consulter cette liste pour vérifier si un certificat est toujours autorisé.

OCSP Online Certificate Status Protocol C'est un protocole qui permet de demander à un service si un certificat particulier est encore valide ou s'il a été révoqué.



RSA Algorithme de cryptographie asymétrique qui permet de sécuriser les échanges de données et de réaliser des signatures numériques.
On parle de cryptographie asymétrique parce que **RSA** utilise deux clés mathématiquement liées, mais différentes :
Clé publique : elle peut être communiquée librement à d'autres personnes. Elle permet notamment de vérifier une signature ou, dans certains usages, de chiffrer des données.
Clé privée : elle doit rester secrète. Elle permet notamment de déchiffrer les données chiffrées avec la clé publique correspondante ou de créer une signature numérique.
RSA peut être utilisé pour générer les clés d'une **autorité de certification** (**CA**), signer des certificats numériques et vérifier des signatures.
RSA repose sur la difficulté mathématique de factoriser de très grands nombres entiers.

ECC **Elliptic Curve Cryptography**. Cryptographie basée sur les courbes elliptiques. C'est une famille de méthodes cryptographiques asymétriques qui utilise les propriétés mathématiques des courbes elliptiques pour créer des clés publiques et privées. **ECC** présente un avantage important : elle permet d'obtenir un niveau de sécurité élevé avec des clés beaucoup plus petites que celles nécessaires avec **RSA**.
ECC permet :

De réduire la taille des clés.
De diminuer les ressources nécessaires aux opérations cryptographiques.
D'améliorer les performances dans certains environnements.

Deux systèmes :

- Un système **RSA** utilisant une clé de 3072 bits.
- Un système **ECC** utilisant une clé de 256 bits.

Ces deux systèmes offrent un niveau de sécurité comparable dans les usages classiques.

ECC est la famille de cryptographie, tandis qu'**ECDSA** est un algorithme de signature numérique appartenant à cette famille.

SHA-256 **Secure Hash Algorithm 256 bits.** **SHA-256** est une fonction de hachage cryptographique appartenant à la famille **SHA-2**.
Son rôle est de transformer des données de n'importe quelle taille en une empreinte numérique de taille fixe de 256 bits (soit 64 caractères hexadécimaux). Cette empreinte est appelée **hash** ou condensat.
Contrairement au chiffrement, le hachage ne permet pas de retrouver les données originales à partir de leur empreinte.

Ses principales utilisations :

- Vérifier l'intégrité des fichiers.
- Participer aux signatures numériques.
- Vérifier l'intégrité des certificats.
- Stocker des empreintes de données.
- Participer à la vérification des signatures cryptographiques.

  Dans une **PKI** SHA-256 est notamment utilisé avec **RSA** ou **ECDSA** pour produire des signatures numériques.

AES **Advanced Encryption Standard.** **AES** est un algorithme de chiffrement symétrique utilisé pour protéger la confidentialité des données. On parle de chiffrement symétrique parce qu'une même clé secrète est utilisée pour chiffrer et déchiffrer les données. Contrairement à **RSA** et **ECC**, **AES** ne fonctionne pas avec une paire de clés publique et privée.
AES propose trois tailles de clés :

- AES-128 : clé de 128 bits.
- AES-192 : clé de 192 bits.
- AES-256 : clé de 256 bits.

Plus la clé est longue, plus le nombre de combinaisons possibles est important.

Ses principales utilisations :

- Chiffrement des fichiers.

- Protection des bases de données.
- Sécurisation des communications réseau.
- Protection des données stockées.
- Chiffrement des communications HTTPS avec TLS.

Dans une **PKI** :
AES peut être utilisé pour chiffrer les communications sécurisées entre un client et un serveur, notamment dans le cadre d'une connexion **TLS**.

TLS 1.3 **Transport Layer Security 1.3** Version moderne du protocole **TLS**. TLS 1.3 utilise des algorithmes comme AES pour chiffrer les communications et SHA-256 pour certaines opérations cryptographiques.

Principaux avantages :

- Une meilleure sécurité que les anciennes versions de TLS.
- Une négociation de connexion plus rapide.
- La suppression de plusieurs anciens algorithmes et mécanismes considérés comme faibles.
- L'utilisation d'échanges de clés modernes, notamment basés sur Diffie-Hellman éphémère.

Lorsqu'on se connectes à un site **HTTPS**, **TLS** combine plusieurs mécanismes cryptographiques.

1. **ECDHE** : permet au navigateur et au serveur d'établir un secret partagé.
2. **ECDSA** ou **RSA** : peut permettre au serveur de prouver son identité grâce à une signature numérique de son échange de clés.
3. **AES** : utilise les clés de session dérivées du secret partagé pour chiffrer les données échangées.

Ainsi, chaque technologie joue un rôle complémentaire.

La politique de certification (CP/CPS) CP Certificate Policy, **CPS** Certification Practice Statement
CP Certificate Policy (politique de certification). Elle explique les règles que nous appliquons.
Par exemple :

- Qui peut obtenir un certificat ?
- Quels types de certificats peuvent être délivrés ?
- Quelle est leur durée de validité ?

CPS Certification Practice Statement. C'est la description des procédures techniques et organisationnelles réellement utilisées pour appliquer cette

politique.
Par exemple :

- Comment générons-nous les clés ?
- Où stockons-nous les clés privées ?
- Comment créons-nous les certificats ?
- Comment révoquons-nous un certificat ?
- Comment protégeons-nous la CA racine ?

Infrastructure **PKI** avec **OpenSSL** (Un logiciel et une bibliothèque utilisés pour les opérations cryptographiques).

Elle comprendra :

1. Une CA racine.
2. Une CA intermédiaire.
3. Des certificats serveur.
4. Des certificats client.
5. Un service web sécurisé en HTTPS.
6. L'authentification mutuelle mTLS.
7. La révocation des certificats avec une CRL.
8. Des tests prouvant que les certificats révoqués sont refusés.

OpenSSL permet de :

- Générer des clés privées et publiques.
- Créer des certificats.
- Signer des certificats.
- Vérifier des certificats.
- Créer des autorités de certification.
- Révoquer des certificats.
- Générer des listes CRL.



Démonstration:
Test 1: ***connexion avec un certificat valide*** -> On ouvre le site web -> Le serveur vérifie le certificat client -> Connexion acceptée.

Test 2 : ***connexion sans certificat*** -> On essaies d'accéder au site sans présenter de certificat client -> Connexion refusée.

Test 3 : ***connexion avec un certificat révoqué*** -> Nous révoquons le certificat d'un utilisateur -> Nous mettons à jour la CRL -> L'utilisateur essaie de se connecter à nouveau -> Connexion refusée.

Choisir les algorithmes cryptographiques:

Élément	Choix	Explication
CA racine	RSA 4096 bits	Protection renforcée de l'autorité principale
CA intermédiaire	RSA 3072 bits	Bon niveau de sécurité
Certificat serveur	RSA 3072 bits	Identification du serveur
Certificat client	RSA 3072 bits	Identification de l'utilisateur
Signature	SHA-256	Vérification de l'intégrité
Protection des clés	AES-256	Protection des fichiers de clés privées
Protocole	TLS 1.2 / 1.3	Sécurisation des communications

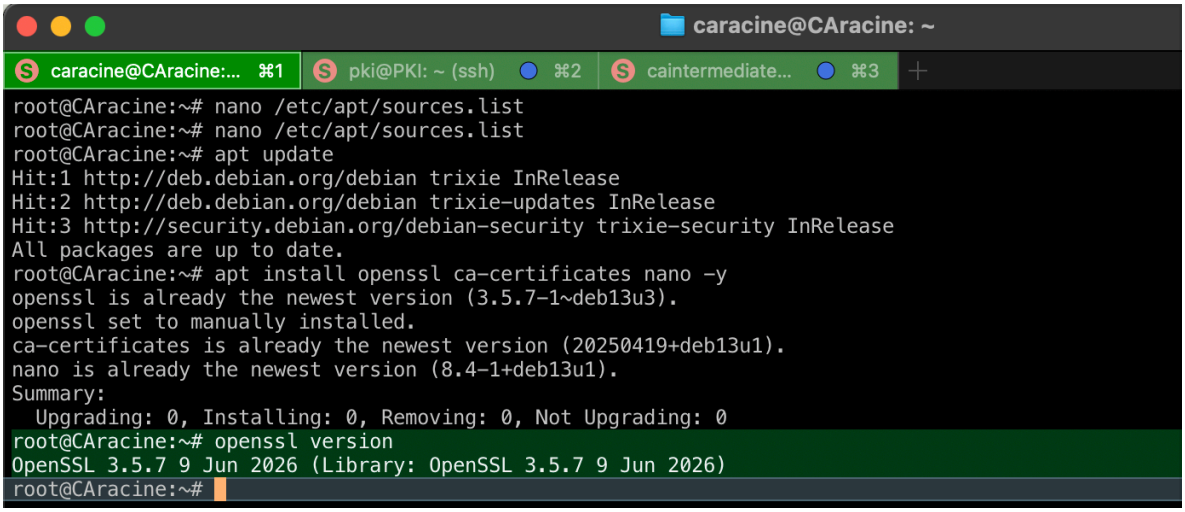
Les noms de notre infrastructure:

Élément	Nom
Entreprise fictive	Entreprise-PKI
CA racine	Entreprise-PKI Root CA
CA intermédiaire	Entreprise-PKI Intermediate CA
Serveur web	pki.entreprise.test

VM1 CAracine

Rôle : créer la CA racine et signer le certificat de la CA intermédiaire.

- openssl : création des clés et certificats.



VM2 CAintermediate

C'est elle qui délivrera les certificats des serveurs et des utilisateurs.

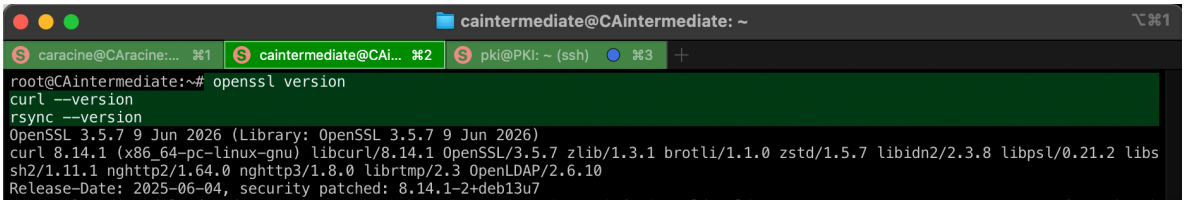
Installer des logiciels:
apt install openssl ca-certificates nano curl rsync -y

```
root@CAintermediate:~# apt install openssl ca-certificates nano curl rsync -y
openssl is already the newest version (3.5.7-1~deb13u3).
openssl set to manually installed.
ca-certificates is already the newest version (20250419+deb13u1).
nano is already the newest version (8.4-1+deb13u1).
Installing:
  curl  rsync

Suggested packages:
  python3-braceexpand

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 0
  Download size: 772 kB
  Space needed: 1,485 kB / 17.4 GB available

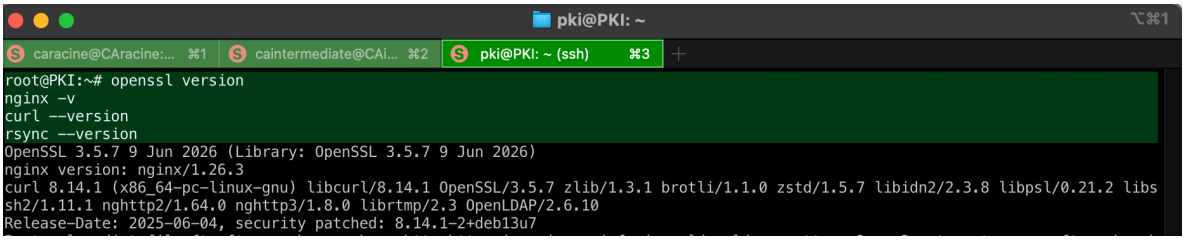
Get:1 http://security.debian.org/debian-security trixie-security/main amd64 rsync amd64 3.5.0+ds1-0+deb13u1 [502 kB]
Get:2 http://security.debian.org/debian-security trixie-security/main amd64 curl amd64 8.14.1-2+deb13u7 [270 kB]
Fetched 772 kB in 0s (4,234 kB/s)
Selecting previously unselected package rsync.
(Reading database ... 39691 files and directories currently installed.)
Preparing to unpack .../rsync_3.5.0+ds1-0+deb13u1_amd64.deb ...
Unpacking rsync (3.5.0+ds1-0+deb13u1) ...
Selecting previously unselected package curl.
Preparing to unpack .../curl_8.14.1-2+deb13u7_amd64.deb ...
Unpacking curl (8.14.1-2+deb13u7) ...
Setting up curl (8.14.1-2+deb13u7) ...
Setting up rsync (3.5.0+ds1-0+deb13u1) ...
rsync.service is a disabled or a static unit, not starting it.
Processing triggers for man-db (2.13.1-1) ...
root@CAintermediate:~#
```



openssl : créer et signer les certificats.
ca-certificates : gérer les autorités de confiance.
curl : tester les connexions.
rsync : transférer les fichiers entre les machines.

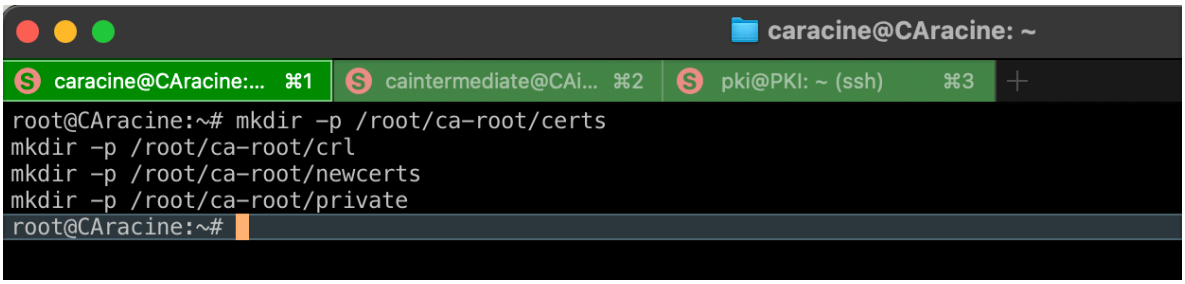
VM3 PKI

Installation des logiciels:



Creation de notre CA racine.

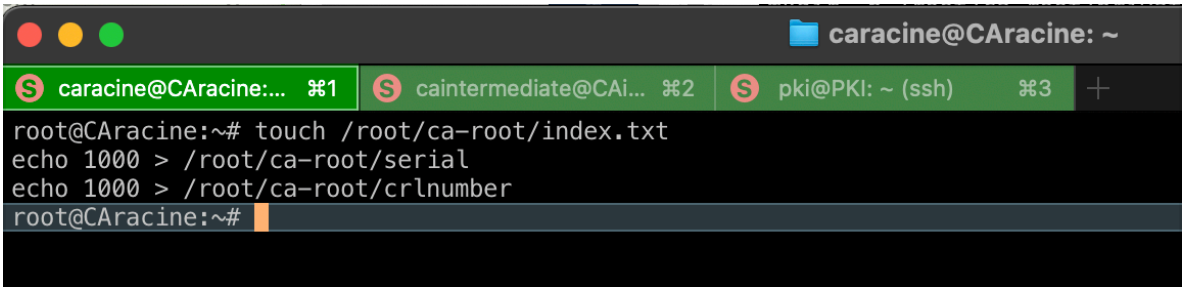
La **CA racine** est l'autorité de confiance principale. Sa clé privée sera protégée par un mot de passe et sa VM sera éteinte après avoir signé le certificat de la **CA intermédiaire**.



Creation des dossiers:

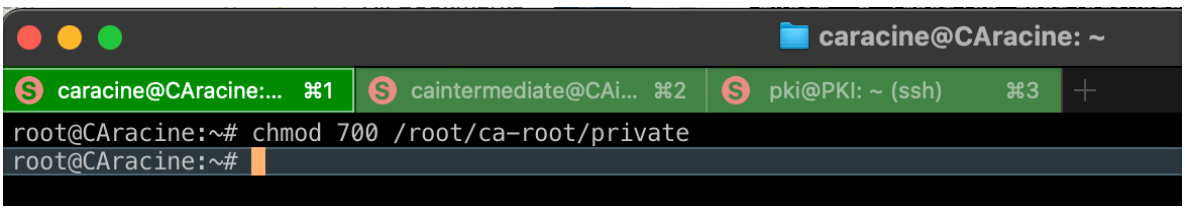
Dossier	Utilité
certs	Certificats publics
crl	Listes de révocation
newcerts	Certificats délivrés
private	Clé privée de la CA racine

Initialisation des fichiers de gestion



Ces fichiers permettent à OpenSSL de suivre les certificats émis, leurs numéros de série et les listes de révocation.

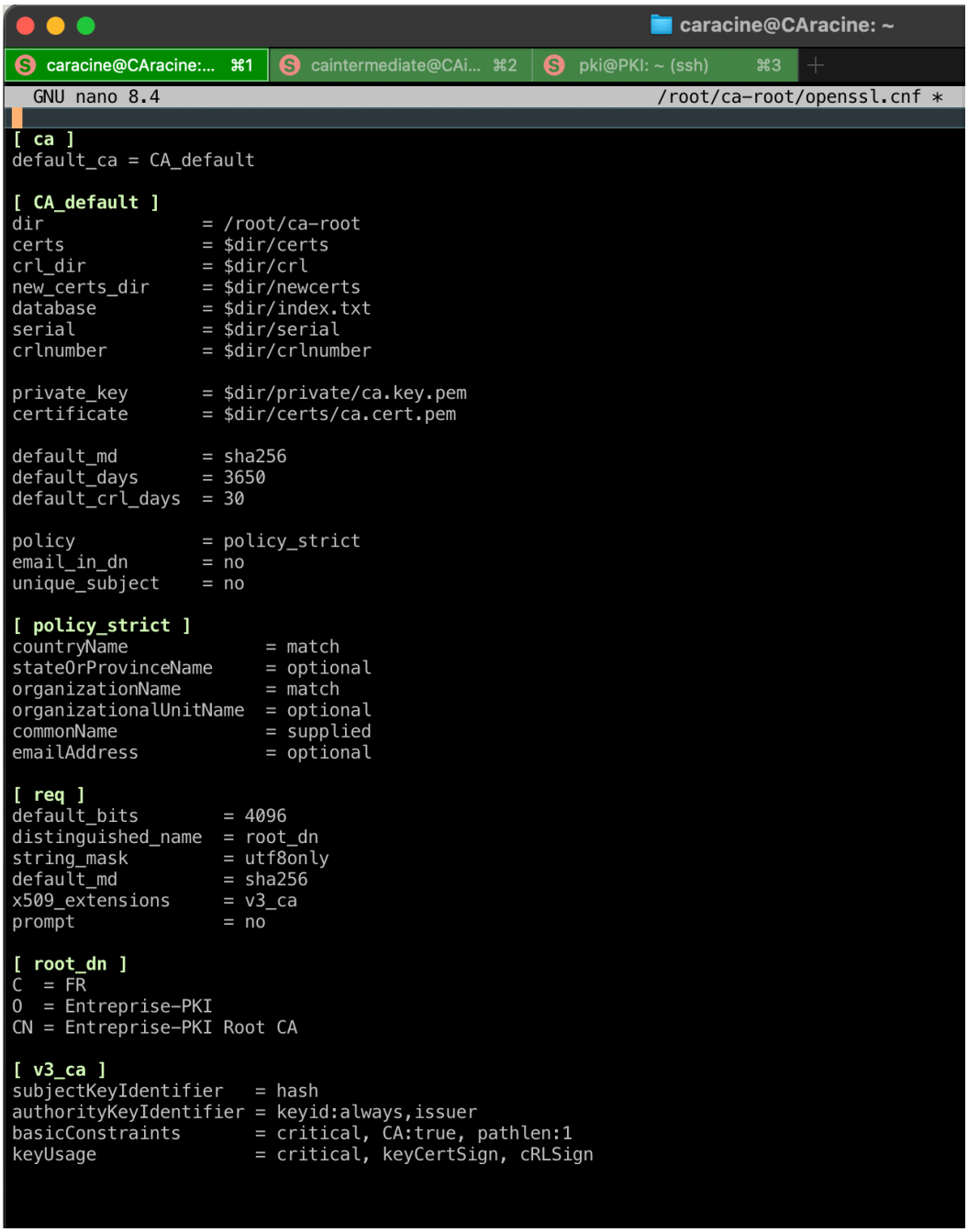
Protéger le dossier privé:



Cela signifie que seul l'administrateur root peut accéder à ce dossier.

Creation du fichier de configuration appelé **openssl.cnf**

nano /root/ca-root/openssl.cnf



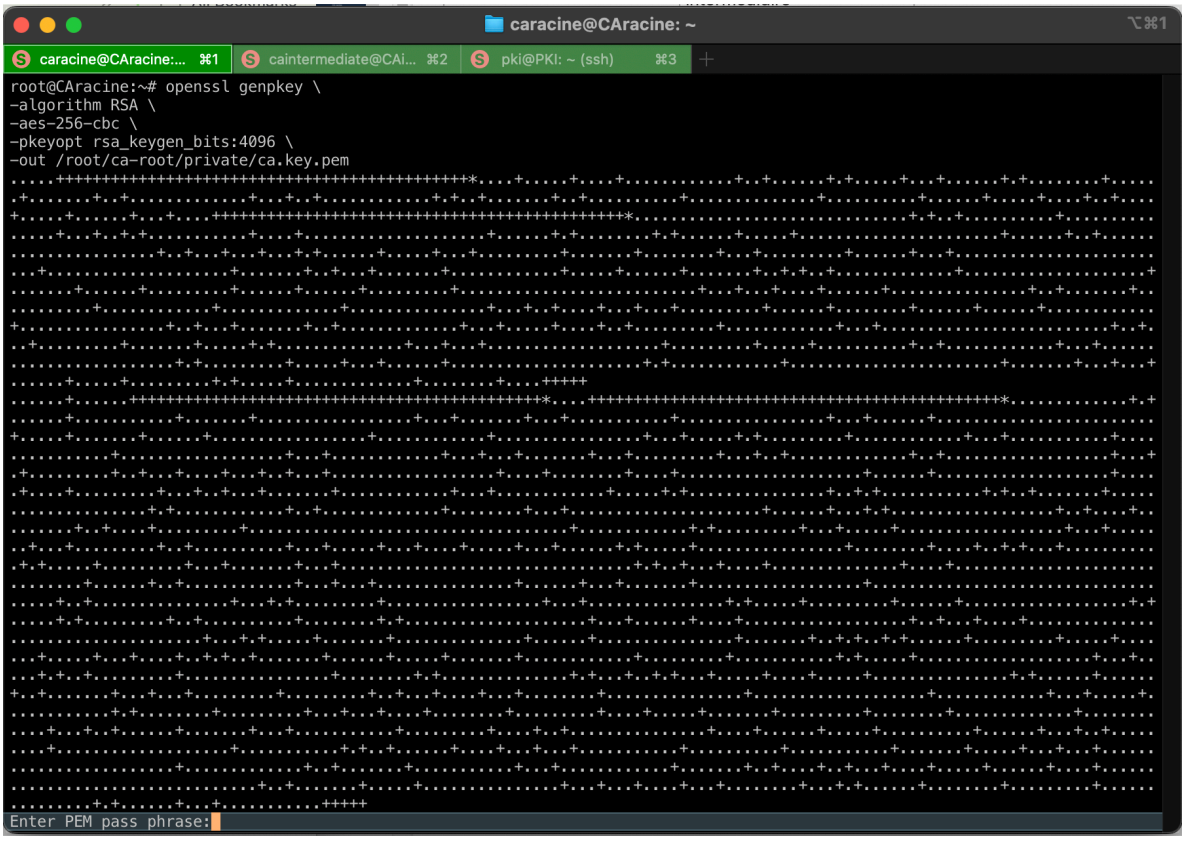
Ce que nous avons configuré:

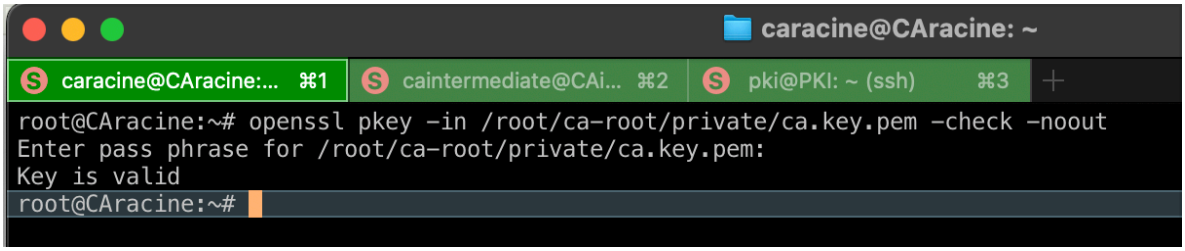
default_md = sha256	Utilisation de SHA-256
default_bits = 4096	Clé RSA de 4096 bits
CA:true	Indique que ce certificat est une autorité

pathlen:1	Autorise un niveau de CA intermédiaire
keyCertSign	Autorise la signature de certificats
cRLSign	Autorise la signature des listes de révocation
default_days = 3650	Durée par défaut de 10 ans

Génération de la clé privée RSA 4096 bits

La clé privée est le secret le plus important de notre **PKI**.
Elle permettra à la **CA racine** de signer le certificat de notre **CA intermédiaire**.



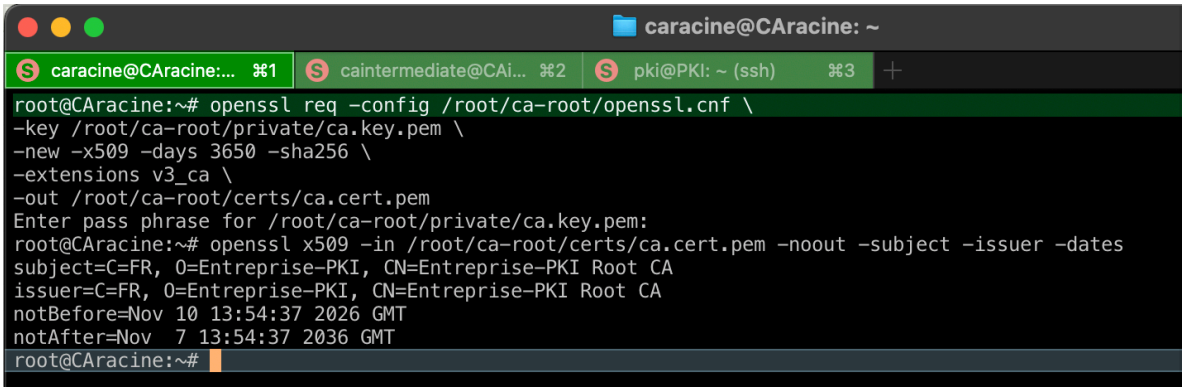


On a une clé RSA de 4096 bits, protégée par AES-256-CBC et accessible uniquement à root.

Creation du certificat racine

Ce certificat sera :

- de type CA (CA:true) ;
- valable 10 ans ;
- signé avec SHA-256 ;
- associé à la clé privée RSA 4096 bits.



CA racine possède maintenant sa clé privée et son certificat numérique.

Pays	FR
Organisation	Entreprise-PKI
Nom	Entreprise-PKI Root CA
Subject	Identique à Issuer
Début de validité	10 novembre 2026
Fin de validité	7 novembre 2036
Durée	10 ans

Le fait que *subject* et *issuer* soient identiques signifie que le certificat est auto-signé.

Vérification des extensions de sécurité

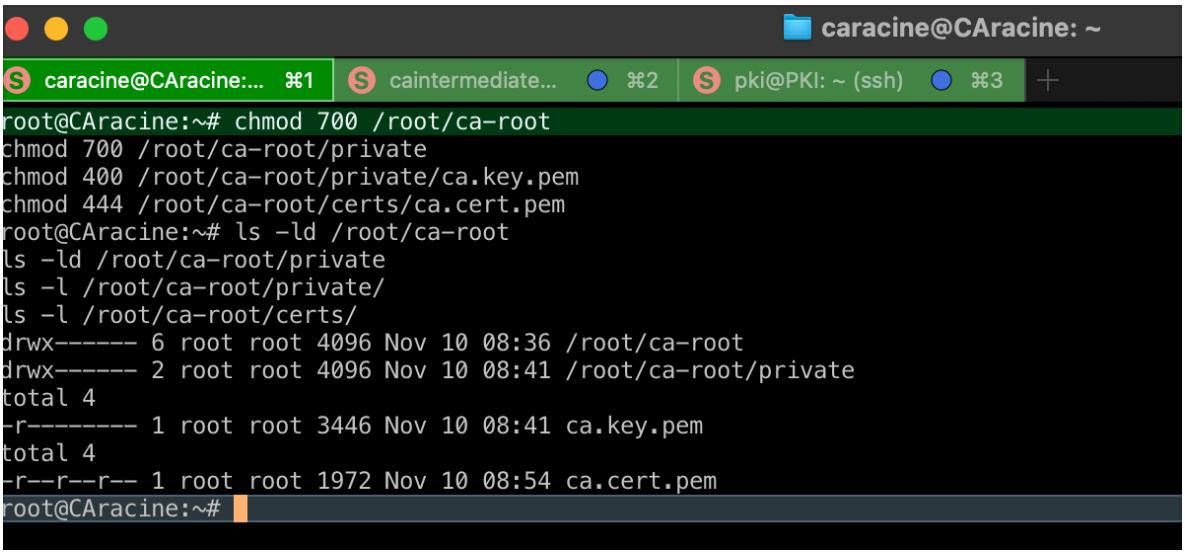
Contrôler que le certificat est bien autorisé à signer d'autres certificats d'autorités de certification:



Version3 -> Certificat X.509 v3
RSA 4096 bits -> Clé de chiffrement robuste
SHA-256 -> Algorithme de signature
CA:TRUE -> Autorité de certification
pathlen:1 -> Une CA intermédiaire autorisée
Certificate Sign -> Peut signer des certificats
CRL Sign -> Peut signer des listes de révocation
Subject = Issuer -> Certificat auto-signé

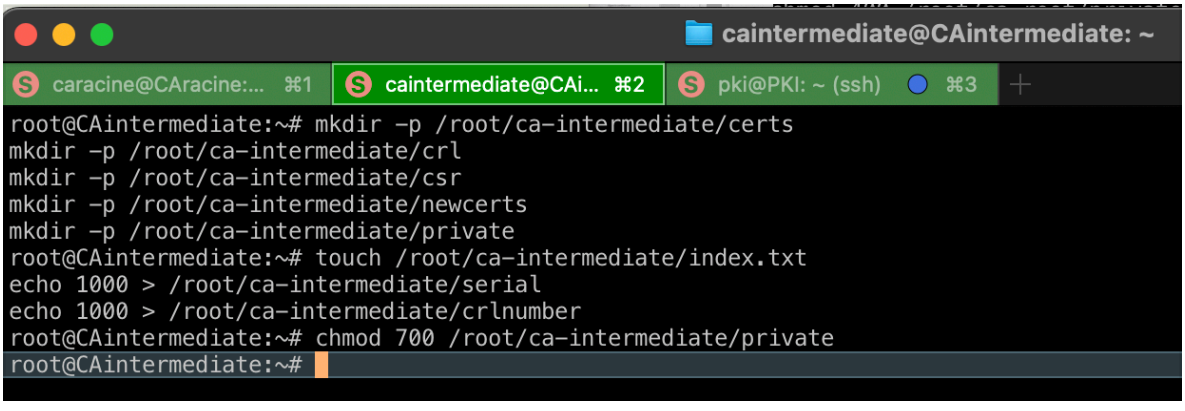
Protéger la CA racine

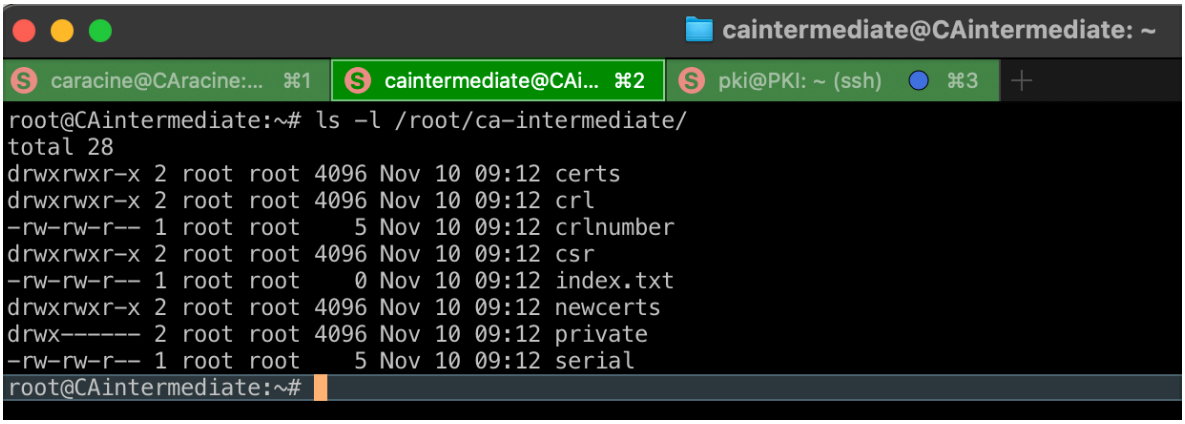
la **CA racine** doit rester hors ligne autant que possible.



VM2 CAintermediate qui délivrera les certificats des serveurs et des utilisateurs.
La **CA intermédiaire** aura pour rôle de signer les certificats des serveurs et des clients. Elle évitera ainsi d'utiliser directement la **CA racine.**

Creation de l'arborescence de la **CA intermédiaire** (créer les dossiers qui accueilleront les certificats, les clés privées et les listes de révocation).





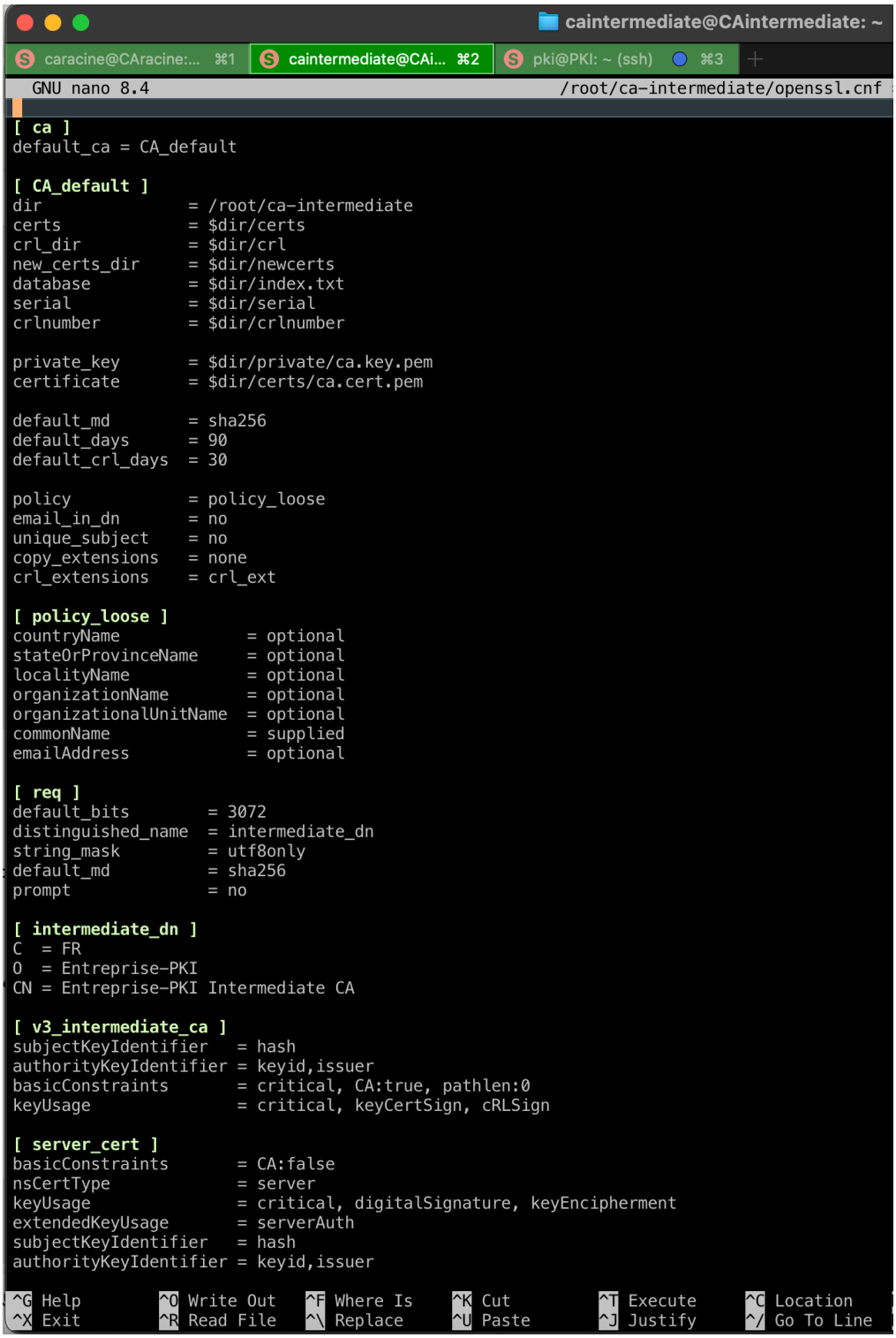
Les dossiers et fichiers nécessaires:

certs : certificat de la CA intermédiaire.
crl : listes de révocation.
csr : demandes de signature de certificat.
newcerts : certificats délivrés.
private : clés privées.
index.txt : registre des certificats délivrés et révoqués.
serial : prochain numéro de série.
crlnumber : numéro de la prochaine liste de révocation.

Creation du fichier openssl.cnf

Ce fichier indique à **OpenSSL** comment gérer les certificats, les numéros de série, les révocations et les signatures:

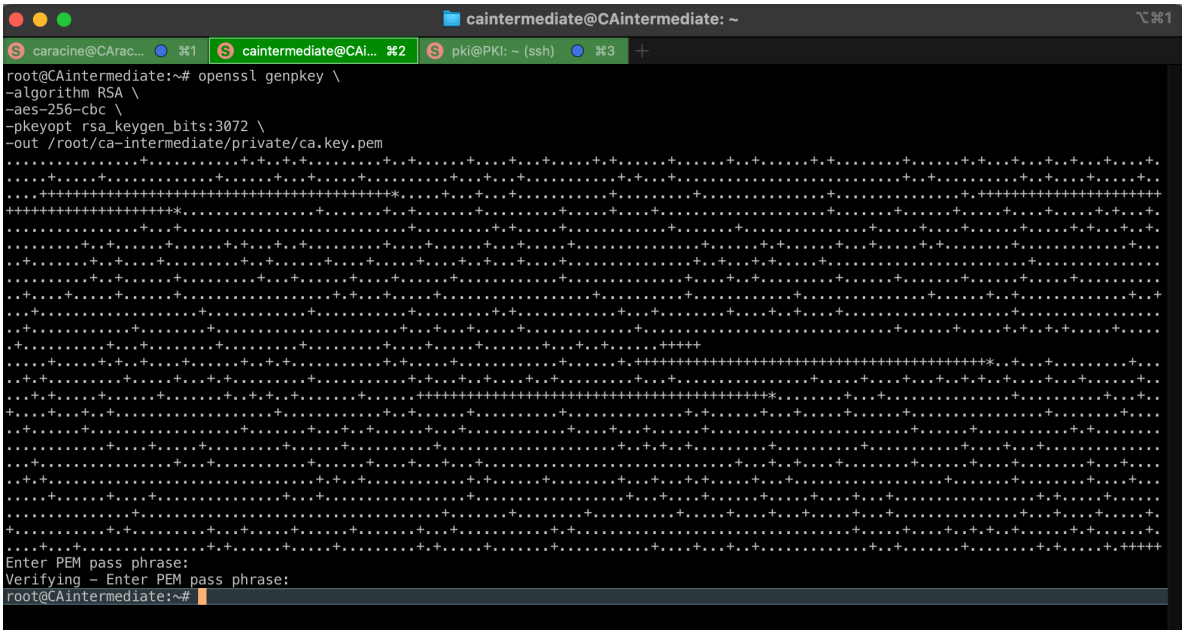
nano /root/ca-intermediate/openssl.cnf

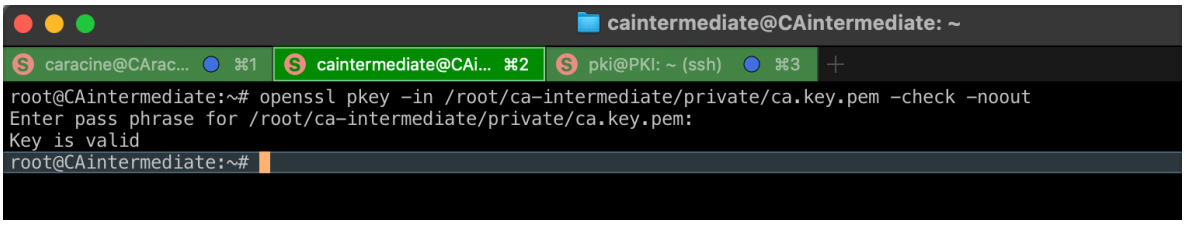


Explication des paramètres:
default_days = 90 : les certificats serveur et client seront valables 90 jours par défaut.
default_crl_days = 30 : les listes de révocation seront valables 30 jours.
CA:true, pathlen:0 : la CA intermédiaire pourra signer des certificats finaux, mais pas créer une autre CA intermédiaire.
serverAuth : usage prévu pour les certificats serveur.
clientAuth : usage prévu pour les certificats client.
copy_extensions = none : évite de recopier automatiquement des extensions

potentiellement dangereuses depuis une CSR.
crl_extensions : permet de signer correctement les listes de révocation.

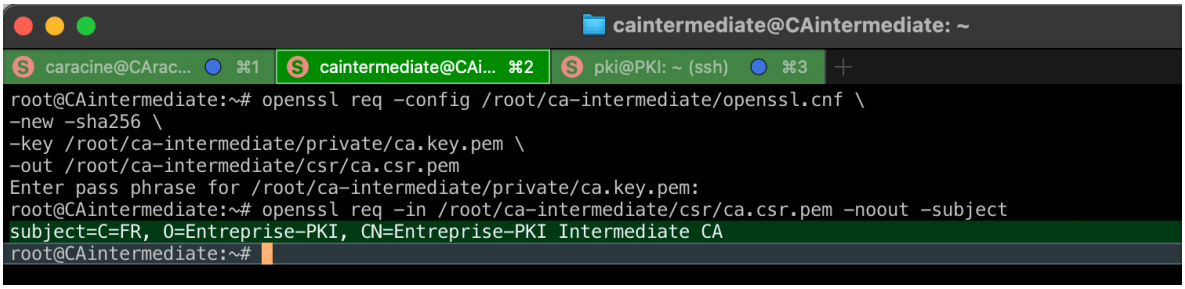
La création de la clé privée de la CA intermédiaire.



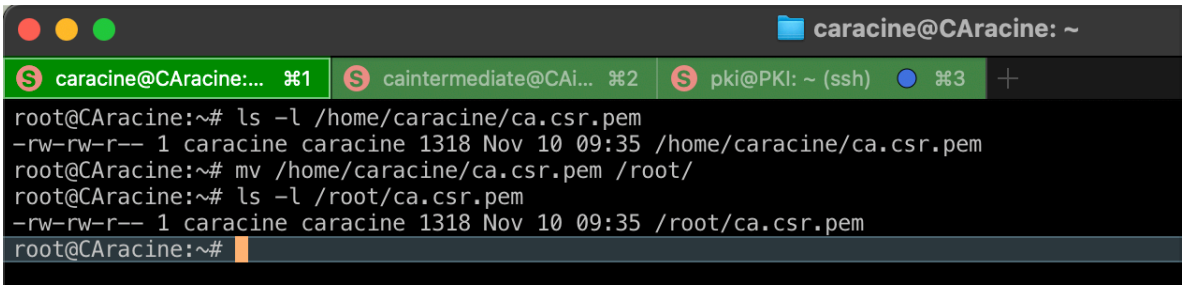


Creationde la demande de certificat (CSR)
La **CA intermédiaire** doit demander à la **CA racine** de lui délivrer un certificat.

Une **CSR (Certificate Signing Request)** est un document qui contient:
- L'identité de la CA intermédiaire.
- Sa clé publique.
- Une demande de signature.
- La CSR ne contient jamais la clé privée.

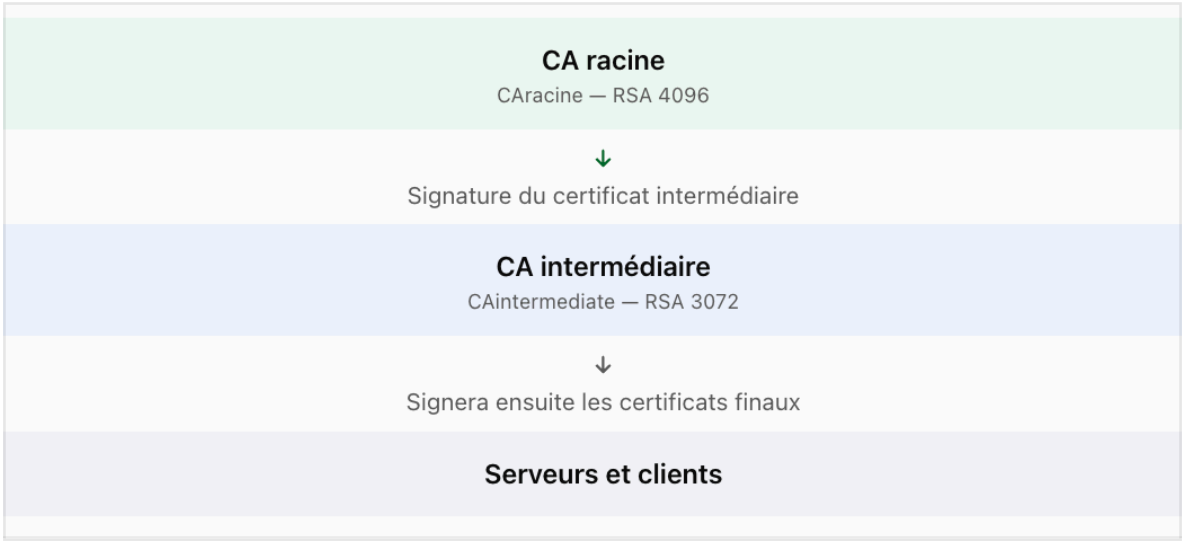


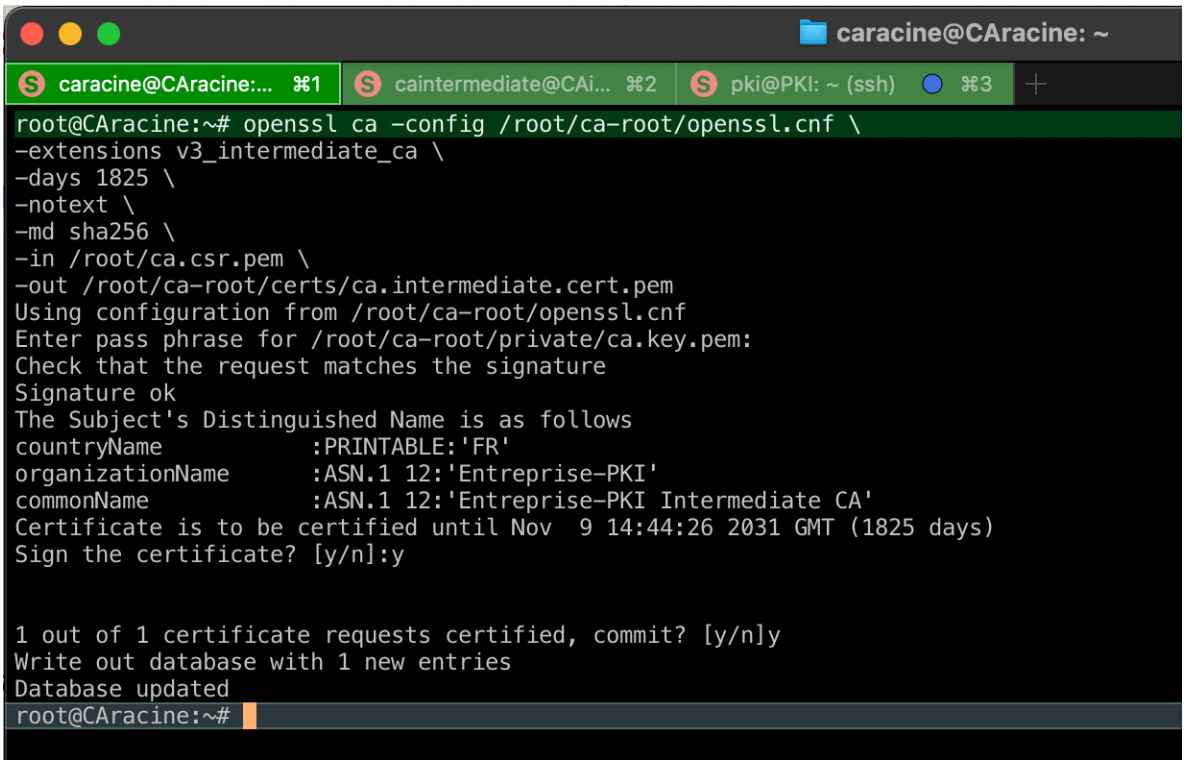
Transférer la **CSR** vers **CAracine**
La CA racine doit recevoir uniquement la CSR, jamais la clé privée de la CA intermédiaire.



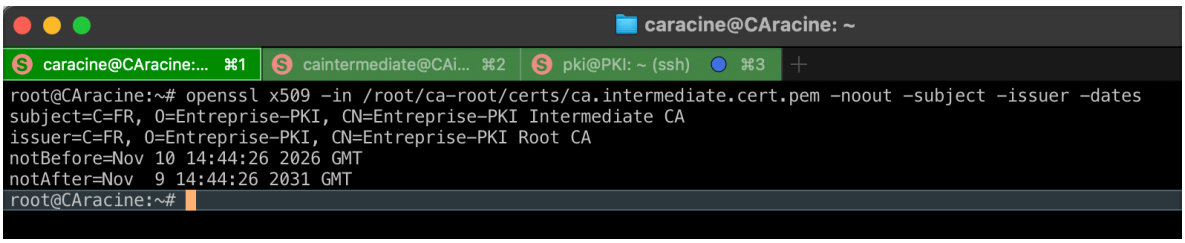
Signer la demande avec la CA racine

(utiliser la clé privée de la CA racine pour signer le certificat de la CA intermédiaire)

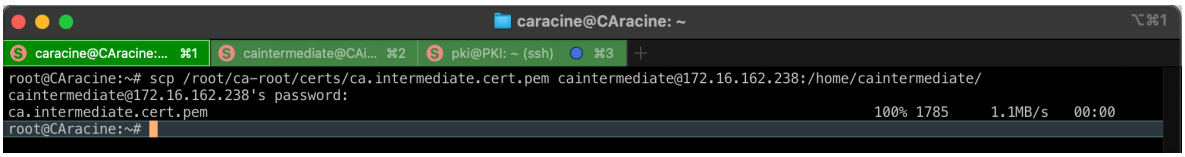




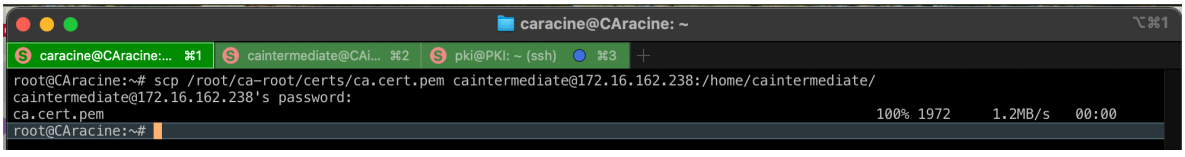
CA racine a signé le certificat de la CA intermédiaire.



Transférer le certificat vers CAintermediate:



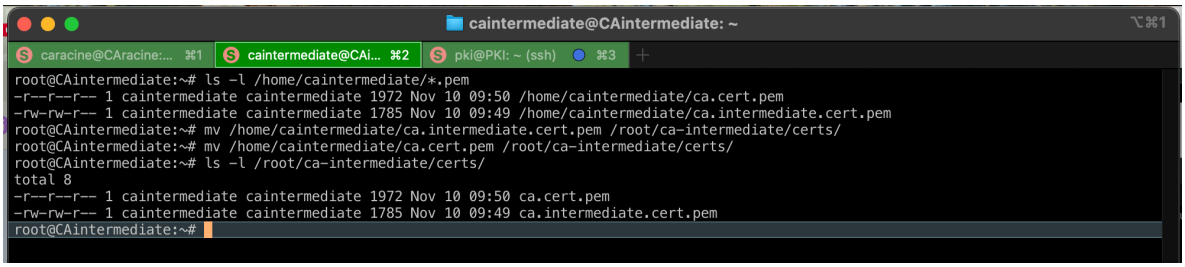
Transférer le certificat public de la CA racine:



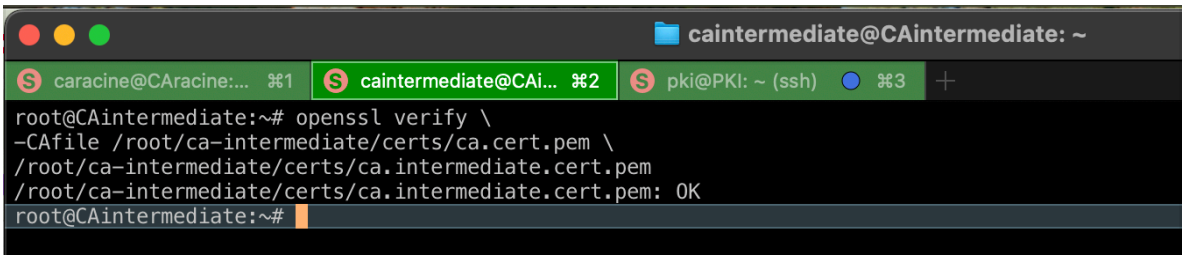
Ce fichier est le certificat public de notre **CA racine**.

Il permettra à la **CA intermédiaire** de vérifier la chaîne de confiance jusqu'à la racine. Nous transférons uniquement *ca.cert.pem* jamais *ca.key.pem*

Les deux certificats sont maintenant au bon emplacement sur CAintermediate:



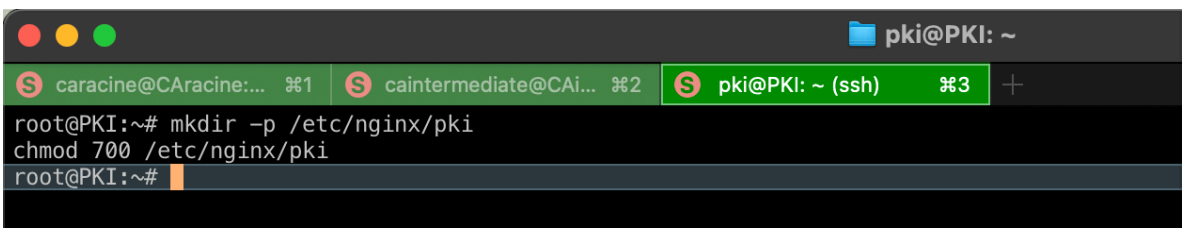
Vérifier la chaîne de confiance:



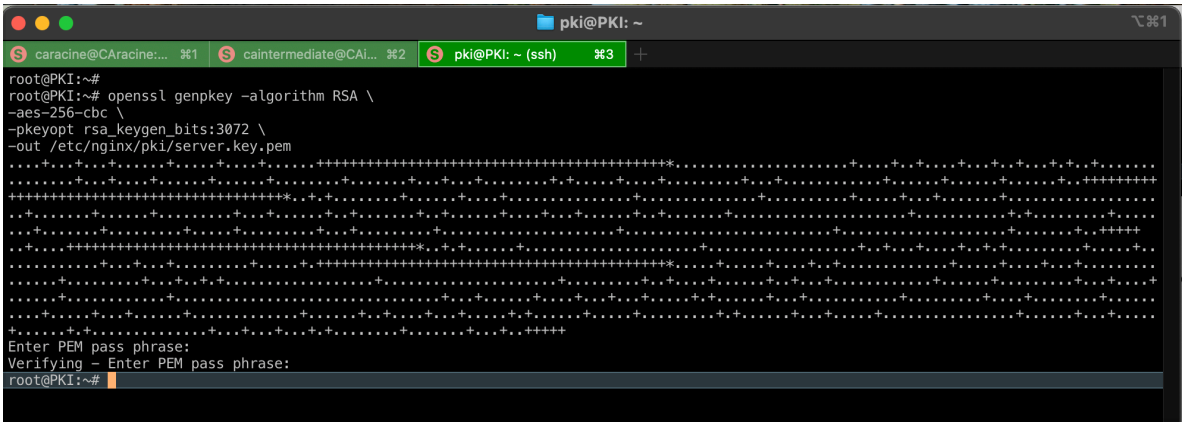
Le certificat de autorité intermédiaire est bien signé par la CA racine et sa signature est valide.

La CA racine peut maintenant être arrêtée et conservée hors ligne : elle a terminé la signature du certificat intermédiaire.

Créer le répertoire des certificats sur PKI sur la VM PKI



Creation de la clé privée du serveur (sur la vm PKI):

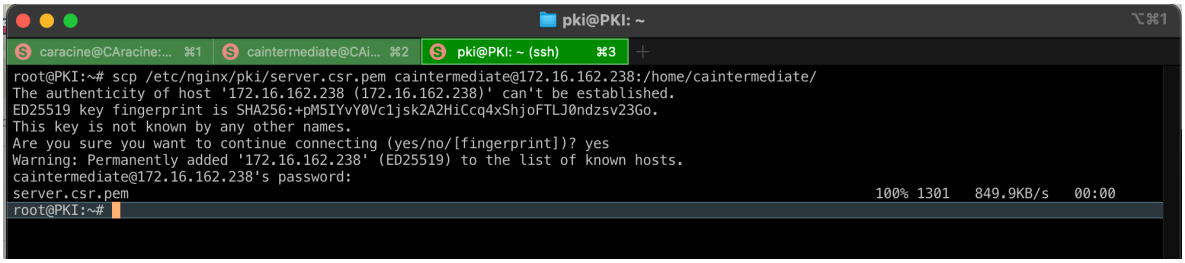


RSA 3072 bits
chiffrée avec AES-256-CBC
protégée par une phrase secrète.

Creation de la demande de certificat (CSR).
La **CSR** est une demande officielle que nous allons envoyer à notre **CA intermédiaire** pour obtenir le certificat HTTPS du serveur.



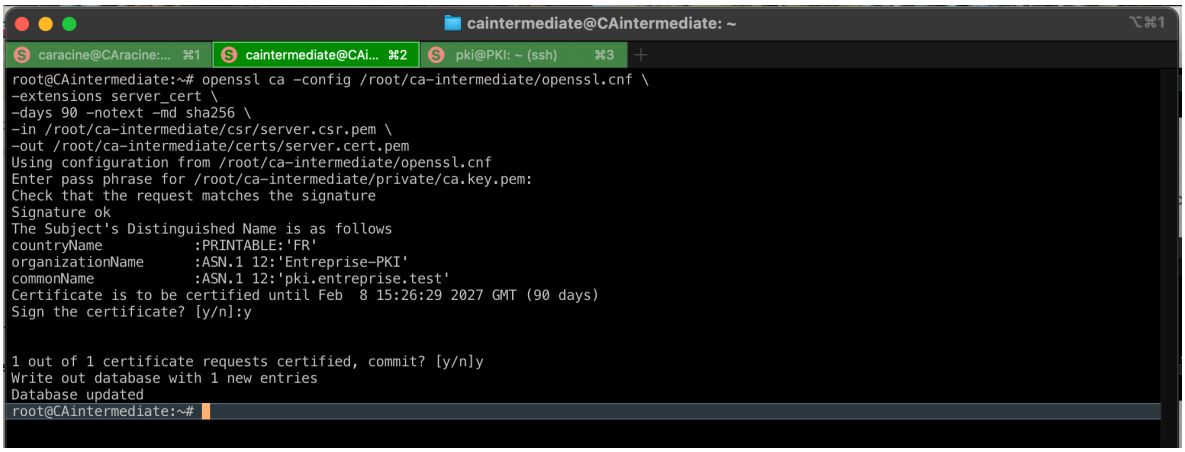
Envoyer le CSR à la CA intermédiaire.(transmettre cette demande à notre autorité intermédiaire CAintermediate, qui va la signer).



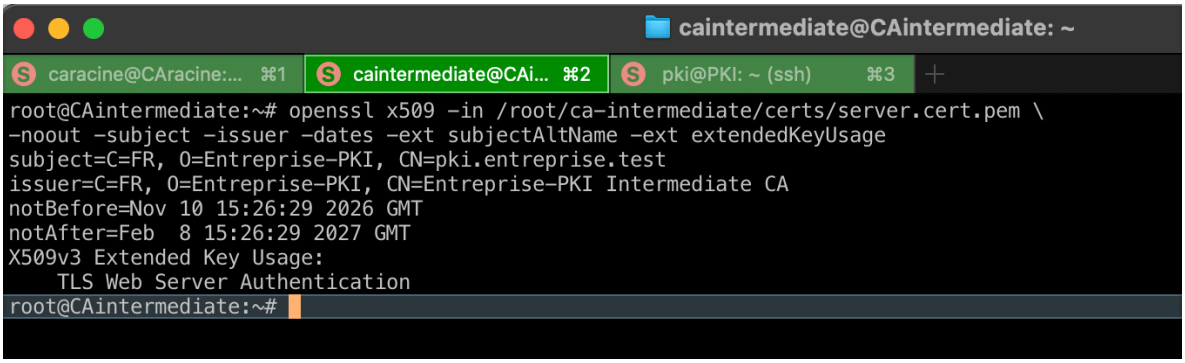
Le fichier server.csr.pem est bien arrivé sur la VM CAintermediate.

Signer le certificat du serveur

Nous allons demander à la CA intermédiaire de signer le CSR que nous avons reçu:

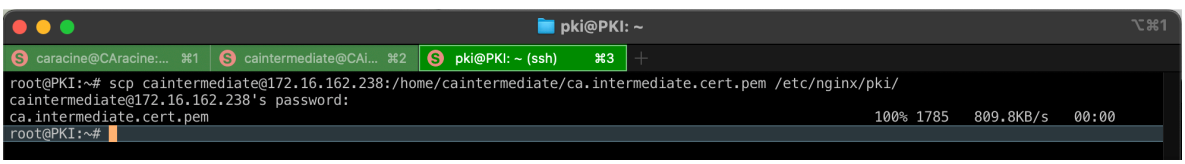


La CA intermédiaire a bien signé le certificat du serveur.
La clé privée du serveur : server.key.pem (reste sur la VM PKI).
La demande CSR : server.csr.pem.
Le certificat signé : server.cert.pem (sur la VM CAintermediate).



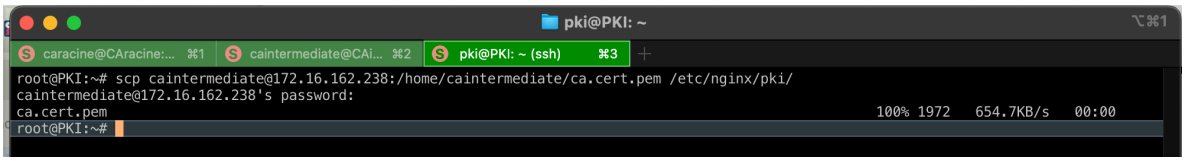
Récupérer le certificat de la CA intermédiaire

Nginx doit disposer du certificat public de la CA intermédiaire pour présenter une chaîne de confiance complète.



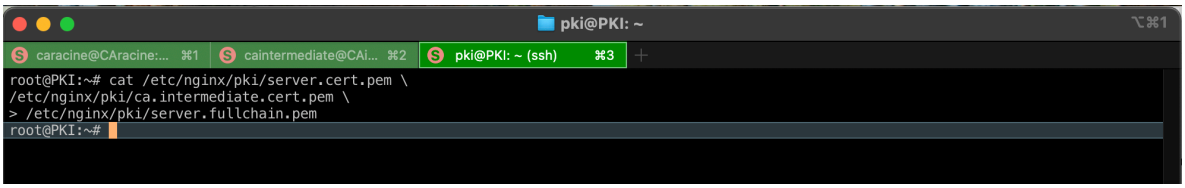
server.key.pem : clé privée du serveur.
server.cert.pem : certificat signé du serveur.
ca.intermediate.cert.pem : certificat public de la CA intermédiaire.

Le certificat racine est public, contrairement à la clé privée de la CA racine, qui doit rester protégée et hors ligne.



La **VM PKI** possède les trois certificats nécessaires :
server.cert.pem : certificat du serveur.
ca.intermediate.cert.pem : certificat de l'autorité intermédiaire.
ca.cert.pem : certificat de l'autorité racine.

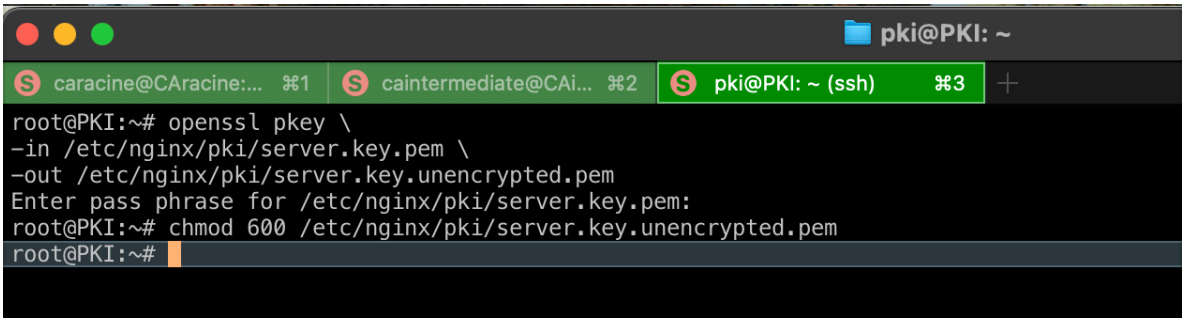
Construire la chaîne complète du serveur
Nous allons créer un fichier qui contient le certificat du serveur suivi de celui de la CA intermédiaire.



Lorsqu'un client se connectera au serveur HTTPS, Nginx lui présentera :
1. Le certificat du serveur.
2. Le certificat de la CA intermédiaire.

Le client pourra ensuite remonter jusqu'au certificat racine qu'il doit déjà considérer comme fiable.

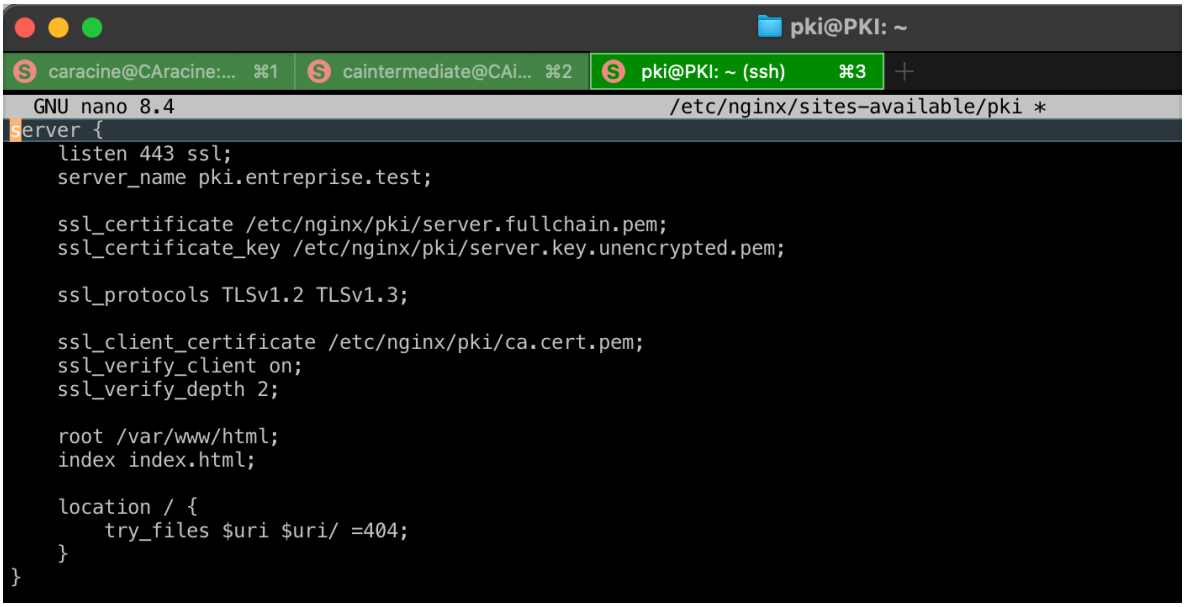
Préparer la clé privée pour Nginx



server.key.pem *: clé originale chiffrée, conservée en sécurité.*
server.key.unencrypted.pem *: copie déchiffrée, réservée à Nginx et accessible uniquement à root.*

Configuration du serveur HTTPS avec Nginx et l'authentification mutuelle (mTLS).

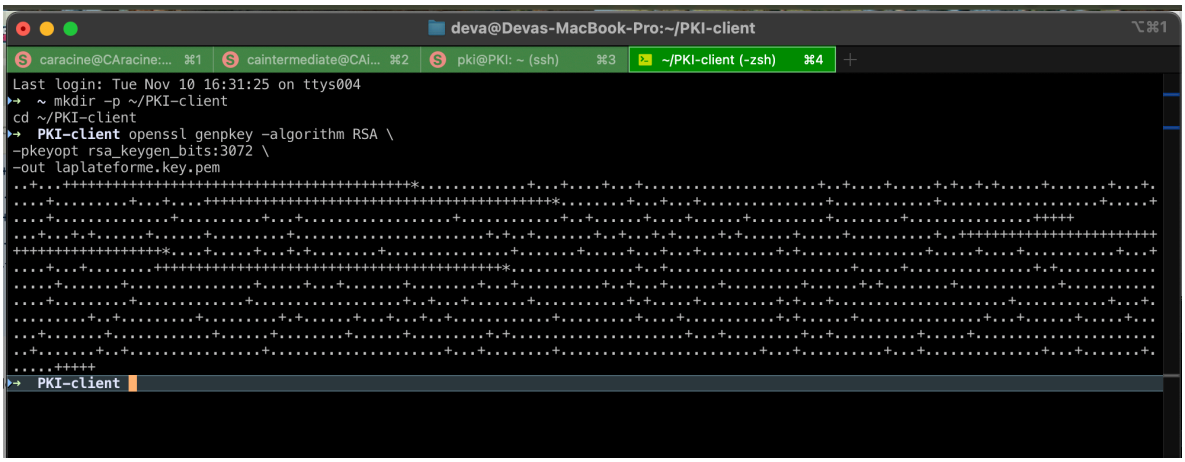
nano /etc/nginx/sites-available/pki



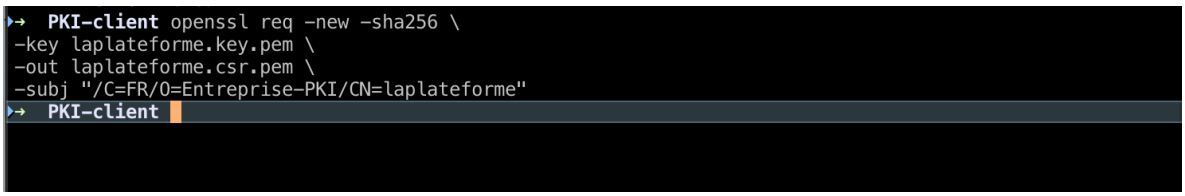
listen 443 ssl -> Active HTTPS sur le port 443
ssl_certificate -> Présente le certificat du serveur et l'intermédiaire
ssl_certificate_key -> Utilise la clé privée du serveur
ssl_client_certificate -> Définit la CA racine de confiance pour les clients
ssl_verify_client on -> Exige un certificat client valide
ssl_verify_depth 2 -> Autorise la chaîne racine → intermédiaire → client



Pour tester **mTLS**, nous avons besoin d'un utilisateur client, que nous appellerons LaPlateforme.
Nous allons générer sa clé privée et sa demande de certificat (**CSR**). La clé privée restera sur le poste client et seule la **CSR** sera envoyée à la **CA intermédiaire**.

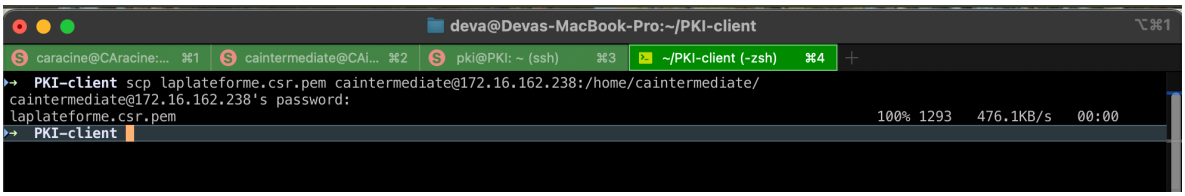


Créer la demande de certificat (CSR)



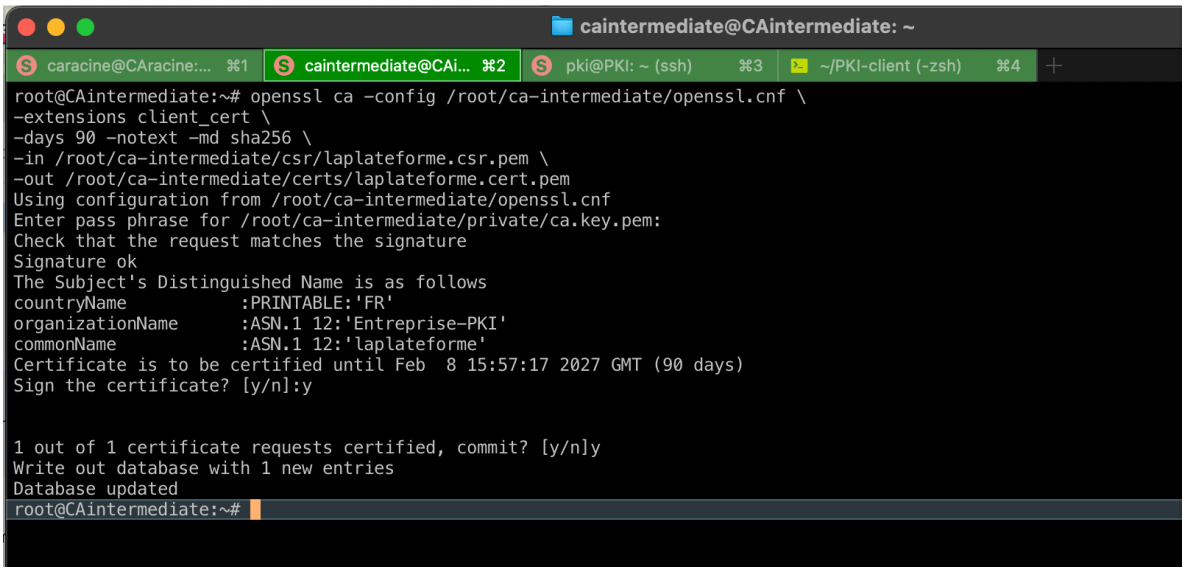
laplateforme.key.pem *: clé privée du client.*
laplateforme.csr.pem *: demande de certificat à transmettre à la CA.*

Envoyer la CSR à CAintermediate



Nous transférons uniquement la CSR, jamais laplateforme.key.pem

Signer le certificat client avec l'extension ***client_cert***, qui donnera au certificat le droit d'être utilisé pour l'authentification d'un client TLS:



*La **CA intermédiaire** a délivré le certificat de laplateforme.*

Le test d'authentification mutuelle (mTLS)

L'objectif sera de démontrer que :

- Un client possédant un certificat valide peut accéder au serveur.
- Un client sans certificat est refusé.

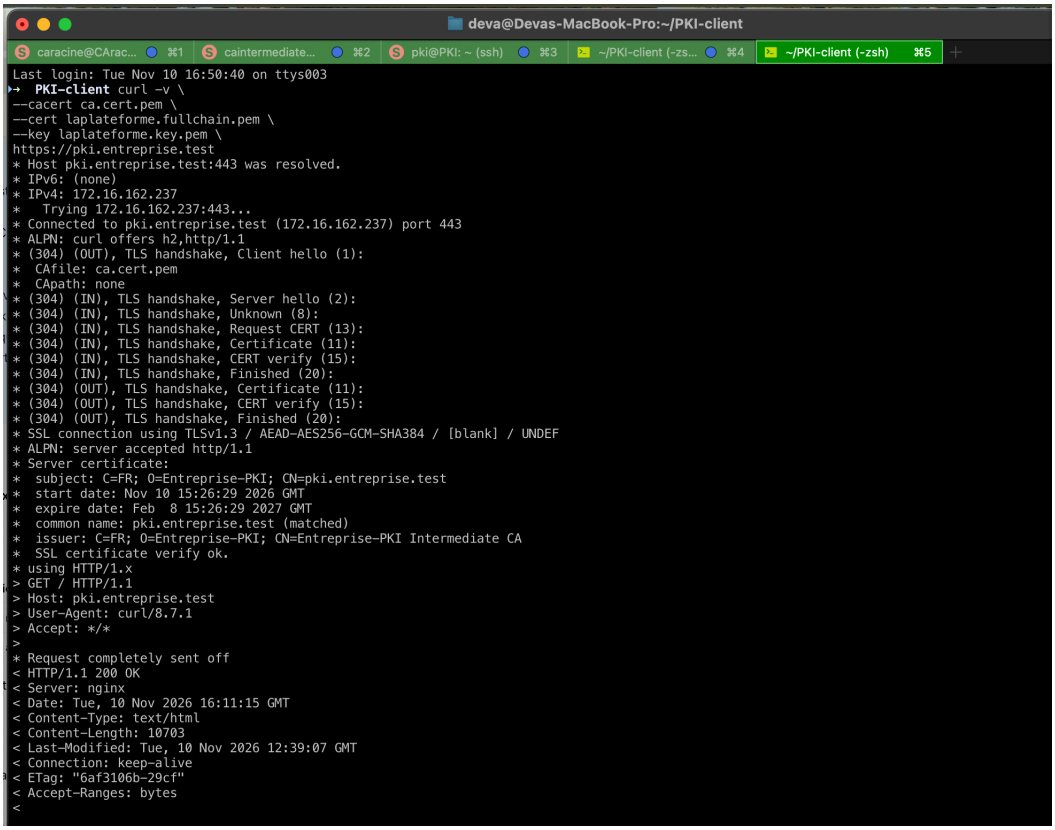
Test 1 : connexion sans certificat client:

Nous allons vérifier que **Nginx** refuse une connexion lorsqu'un client ne présente aucun certificat.



Connexion au serveur -> Réussie -> Le serveur est joignable
Certificat serveur -> Valide -> La CA racine est reconnue
TLS 1.3 -> Activé -> Connexion chiffrée
Certificat client -> Absent -> Le client n'a pas présenté de certificat
Réponse HTTP -> 400 Bad Request -> **Nginx refuse l'accès**

Test 2 connexion avec le certificat valide:
Nous allons vérifier qu'un client possédant un certificat signé par notre CA intermédiaire peut accéder au serveur.



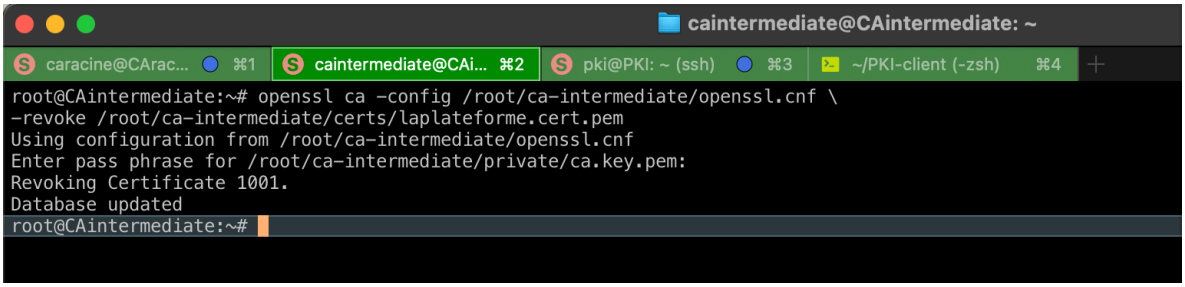
Le certificat client est reconnu.
Sa chaîne de confiance est acceptée.
La clé privée correspond au certificat.
Nginx autorise l'accès.
L'authentification mutuelle TLS (mTLS) fonctionne.

Test	Résultat	
Sans certificat client	HTTP 400	Accès refusé : certificat obligatoire
Avec le certificat laplateforme	HTTP 200	Accès autorisé : certificat reconnu

Le navigateur ou le client ne se contente donc pas de vérifier l'identité du serveur : le serveur vérifie aussi l'identité du client grâce au certificat délivré par l'autorité intermédiaire.

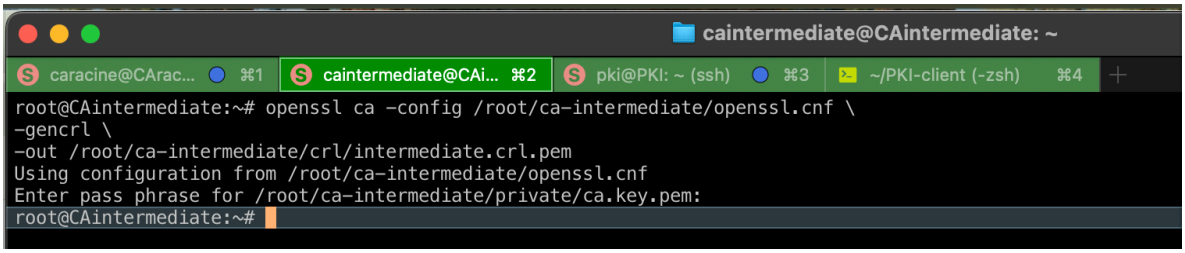
Révoquer le certificat client.

Un certificat valide au départ peut être révoqué par l'autorité intermédiaire, puis refusé par Nginx.

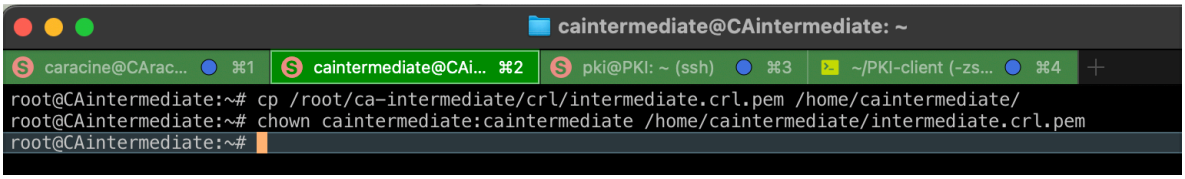


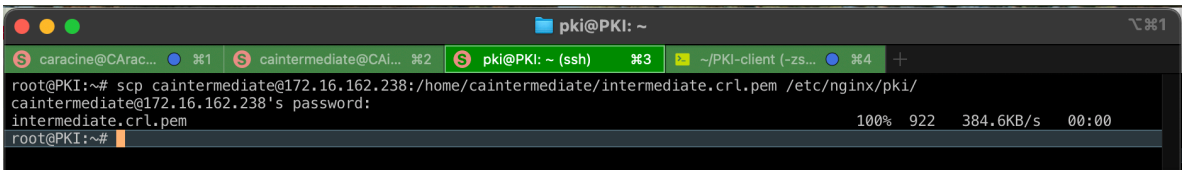
Le certificat *laplateforme* est révoqué dans la base de données de notre autorité intermédiaire

Générer la **CRL** (**Certificate Revocation List**) -la liste officielle des certificats révoqués.



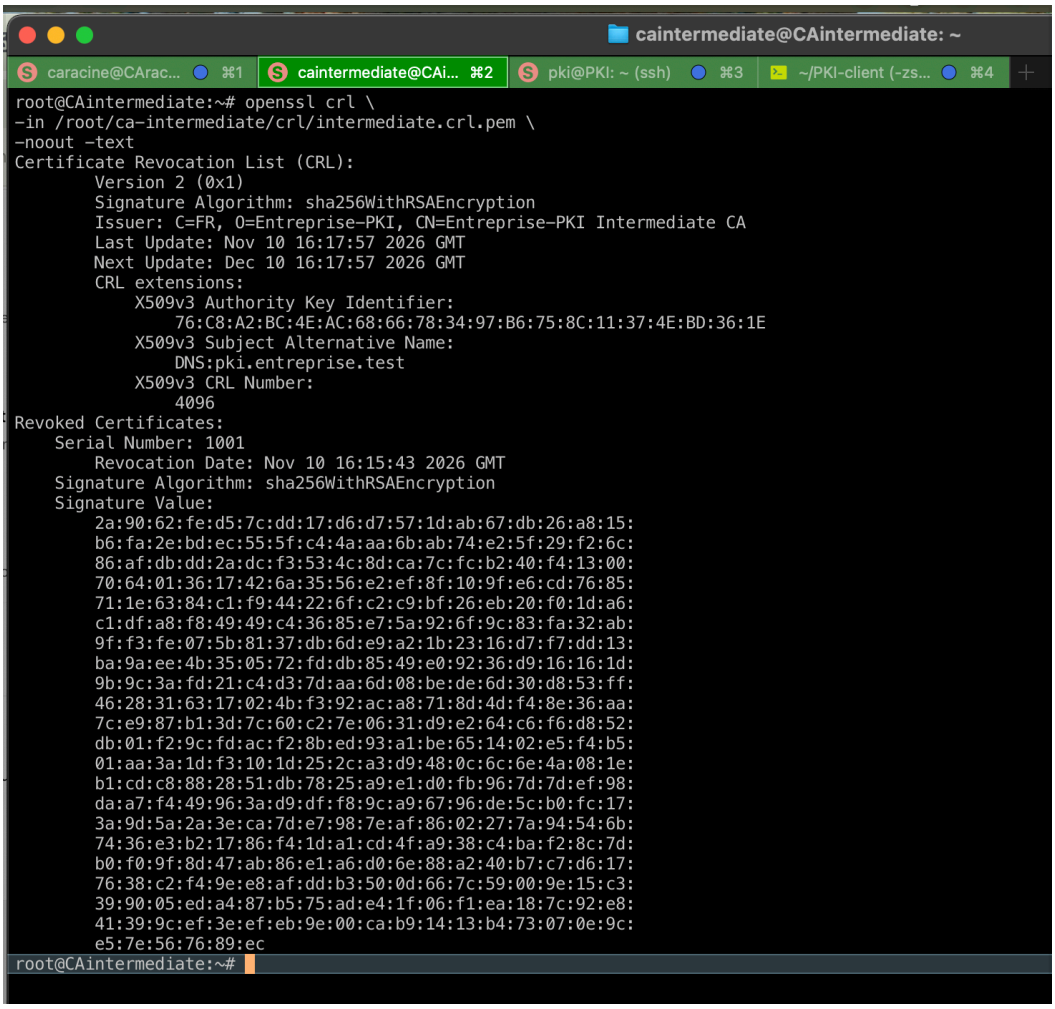
Transférer cette liste de révocation vers la VM PKI, où se trouve Nginx:





Test 3: le certificat révoqué:

```
➜  PKI-client curl -v \
--cacert ca.cert.pem \
--cert laplateforme.fullchain.pem \
--key laplateforme.key.pem \
https://pki.entreprise.test
* Host pki.entreprise.test:443 was resolved.
* IPv6: (none)
* IPv4: 172.16.162.237
*   Trying 172.16.162.237:443...
* Connected to pki.entreprise.test (172.16.162.237) port 443
* ALPN: curl offers h2,http/1.1
* (304) (OUT), TLS handshake, Client hello (1):
*  CAfile: ca.cert.pem
*  CApath: none
* (304) (IN), TLS handshake, Server hello (2):
* (304) (IN), TLS handshake, Unknown (8):
* (304) (IN), TLS handshake, Request CERT (13):
* (304) (IN), TLS handshake, Certificate (11):
* (304) (IN), TLS handshake, CERT verify (15):
* (304) (IN), TLS handshake, Finished (20):
* (304) (OUT), TLS handshake, Certificate (11):
* (304) (OUT), TLS handshake, CERT verify (15):
* (304) (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / AEAD-AES256-GCM-SHA384 / [blank] / UNDEF
* ALPN: server accepted http/1.1
* Server certificate:
*  subject: C=FR; O=Entreprise-PKI; CN=pki.entreprise.test
*  start date: Nov 10 15:26:29 2026 GMT
*  expire date: Feb  8 15:26:29 2027 GMT
*  common name: pki.entreprise.test (matched)
*  issuer: C=FR; O=Entreprise-PKI; CN=Entreprise-PKI Intermediate CA
*  SSL certificate verify ok.
* using HTTP/1.x
> GET / HTTP/1.1
> Host: pki.entreprise.test
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 400 Bad Request
< Server: nginx
< Date: Tue, 10 Nov 2026 16:27:06 GMT
< Content-Type: text/html
< Content-Length: 208
< Connection: close
<
<html>
<head><title>400 The SSL certificate error</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<center>The SSL certificate error</center>
<hr><center>nginx</center>
</body>
</html>
* Closing connection
➜  PKI-client
```



Sans certificat client -> HTTP 400 -> *Accès refusé*
Avec certificat valide avant révocation -> HTTP 200 -> *Accès autorisé*
Avec certificat après révocation -> HTTP 400 – SSL certificate error -> *Accès refusé*

Nginx a accepté le certificat laplateforme avant sa révocation. Après la révocation par l'autorité intermédiaire et l'installation de la **CRL**, **Nginx** refuse désormais ce même certificat.

On as donc réalisé et testé les trois étapes essentielles :

1. ***Émission d'un certificat client par l'autorité intermédiaire.***
2. ***Révocation de ce certificat et génération de la CRL.***
3. ***Refus du certificat révoqué par Nginx, avec une réponse HTTP 400.***